

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Delhi-NCR Campus, Modinagar, Ghaziabad

Department of Computer Science & Engineering

BIG DATA ESSENTIALS LABORATORY MANUAL

Hadoop Standalone Mode — 15 Complete, Execution-Ready Practicals

Course: Big Data Essentials

Academic Year: 2026–27 (Odd Semester)

Lab File

Submitted To:
Mr. Tapas Kumar Mishra

Submitted By:

Roll Number: RA2411028030002
Name: Akshay Pratap Singh

Table of Contents

Part A — Important Hadoop Commands 3

Part B — 15 Complete Practical Exercises 5

Practical 1 — Verify Hadoop Standalone Installation and Run a Basic Hadoop Command 5

Practical 2 — Perform Complete File and Directory Operations Using Hadoop's Filesystem Shell 7

Practical 3 — Execute Word Count Using Hadoop's Built-in MapReduce Example 11

Practical 4 — Understand and Demonstrate MapReduce Mapper, Shuffle/Sort, and Reducer Stages 13

Practical 5 — Run Hadoop's Built-in Examples — Grep and Pi Estimation 15

Practical 6 — Run a Custom Java MapReduce Word Count Program 17

Practical 7 — Count the Number of Lines, Words, and Characters in a File 20

Practical 8 — Find the Frequency of Each Character Using MapReduce 23

Practical 9 — Find the Maximum Marks Among Students Using MapReduce 26

Practical 10 — Calculate the Average Marks of Students Using MapReduce 29

Practical 11 — Sort Student Records by Marks Using MapReduce 32

Practical 12 — Filter Student Records Using MapReduce (Map-Only Job) 35

Practical 13 — Count the Number of Students in Each Department Using MapReduce 38

Practical 14 — Compare MapReduce Execution with a Normal Sequential Program 41

Practical 15 — Mini Project — Student Result Analysis Using MapReduce 43

Part C — Optional Transition from Standalone to Pseudo-Distributed Mode 47

Part D — Basic HDFS Commands After Pseudo-Distributed Configuration 48

Part E — Suggested Practical Sequence for Engineering Students 49

Part F — Common Errors and Fixes 50

Part A — Important Hadoop Commands

This quick-reference table lists the Hadoop filesystem-shell and command-line utilities used throughout this manual.

Command	Purpose	Example
hadoop version	Displays the installed Hadoop version	hadoop version
hadoop classpath	Displays Hadoop's Java classpath	hadoop classpath
hadoop fs -ls	Lists files/directories	hadoop fs -ls /
hadoop fs -mkdir	Creates a directory	hadoop fs -mkdir /student
hadoop fs -mkdir -p	Creates parent directories as needed	hadoop fs -mkdir -p /data/input
hadoop fs -touchz	Creates a new empty file	hadoop fs -touchz /student/empty.txt
hadoop fs -put	Uploads a local file into the Hadoop filesystem	hadoop fs -put data.txt /student
hadoop fs -copyFromLocal	Same purpose as -put (upload)	hadoop fs -copyFromLocal data.txt /student
hadoop fs -get	Downloads a file to local storage	hadoop fs -get /student/data.txt .
hadoop fs -copyToLocal	Same purpose as -get (download)	hadoop fs -copyToLocal /student/data.txt .
hadoop fs -cat	Displays file contents	hadoop fs -cat /student/data.txt
hadoop fs -cp	Copies files/directories within the filesystem	hadoop fs -cp /student/a.txt /student/b.txt
hadoop fs -mv	Moves or renames files/directories	hadoop fs -mv /student/a.txt /student/b.txt
hadoop fs -rm	Deletes a file	hadoop fs -rm /student/a.txt
hadoop fs -rm -r	Deletes a directory recursively	hadoop fs -rm -r /student
hadoop fs -du -h	Shows file/directory sizes (human-readable)	hadoop fs -du -h /student
hadoop fs -df -h	Shows filesystem free/used space	hadoop fs -df -h
hadoop fs -count	Counts directories, files, and content size	hadoop fs -count /student
hadoop fs -appendToFile	Appends local file content to a target file	hadoop fs -appendToFile extra.txt /student/data.txt
hadoop fs -tail	Displays the last part of a file	hadoop fs -tail /student/data.txt
hadoop fs -stat	Displays file status information	hadoop fs -stat /student/data.txt
hadoop fs -checksum	Displays a file checksum	hadoop fs -checksum /student/data.txt
hadoop fs -chmod	Changes file permissions	hadoop fs -chmod 755 /student/data.txt
hadoop jar	Runs a Java JAR application with Hadoop	hadoop jar app.jar

hadoop fs vs. hdfs dfs

hdfs dfs is the HDFS-specific shell and requires HDFS daemons (NameNode/DataNode) to be running. hadoop fs is the general-purpose filesystem shell and works against whatever filesystem is configured as default — the local filesystem (file:///) in standalone mode, or HDFS once Pseudo-Distributed Mode is configured (see Part C and Part D). Every command used in Practicals 1–15 uses hadoop fs so that it works correctly in standalone mode; the HDFS-only equivalents are given separately in Part D.

Practical 1 — Verify Hadoop Standalone Installation and Run a Basic Hadoop Command

Aim

To verify the Hadoop installation on a standalone machine and confirm that Hadoop can execute a local MapReduce application.

Learning Objectives

- Verify that Java and Hadoop are correctly installed.
- Check the Hadoop version and the HADOOP_HOME environment variable.
- Locate and run Hadoop's bundled WordCount example.
- Confirm that Hadoop can execute a MapReduce job in standalone mode.

Requirements

- Java (JDK 8/11) installed
- Hadoop 3.x installed and extracted
- Terminal access

Procedure — Commands to Execute

Step 1: Check the Java installation

```
java -version
```

Sample output:

```
openjdk version "11.0.x" 2024-xx-xx
OpenJDK Runtime Environment (build 11.0.x+x)
OpenJDK 64-Bit Server VM (build 11.0.x+x, mixed mode)
```

Step 2: Check the Hadoop version

```
hadoop version
```

Sample output:

```
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git ...
Compiled by ...
```

Step 3: Check the HADOOP_HOME environment variable

```
echo $HADOOP_HOME
echo $PATH | tr ':' '\n' | grep -i hadoop
```

Note: If HADOOP_HOME is empty, edit ~/.bashrc to export HADOOP_HOME and add $HADOOP_HOME/bin and $HADOOP_HOME/sbin to PATH, then run 'source ~/.bashrc'.

Step 4: Create a working directory for this practical

```
mkdir -p ~/hadoop-lab/practical1/input
```

```
cd ~/hadoop-lab/practical1
```

Step 5: Create a test input file

```
echo "Hadoop standalone mode practical" > input/input.txt
cat input/input.txt
```

Step 6: Locate the Hadoop MapReduce examples JAR

```
find "$HADOOP_HOME" -name "hadoop-mapreduce-examples-*.jar"
```

Sample output:

```
/usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
```

Step 7: Run the built-in WordCount example

Replace the JAR path below with the exact path returned in the previous step.

```
hadoop jar /usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
wordcount input output
```

Step 8: View the result

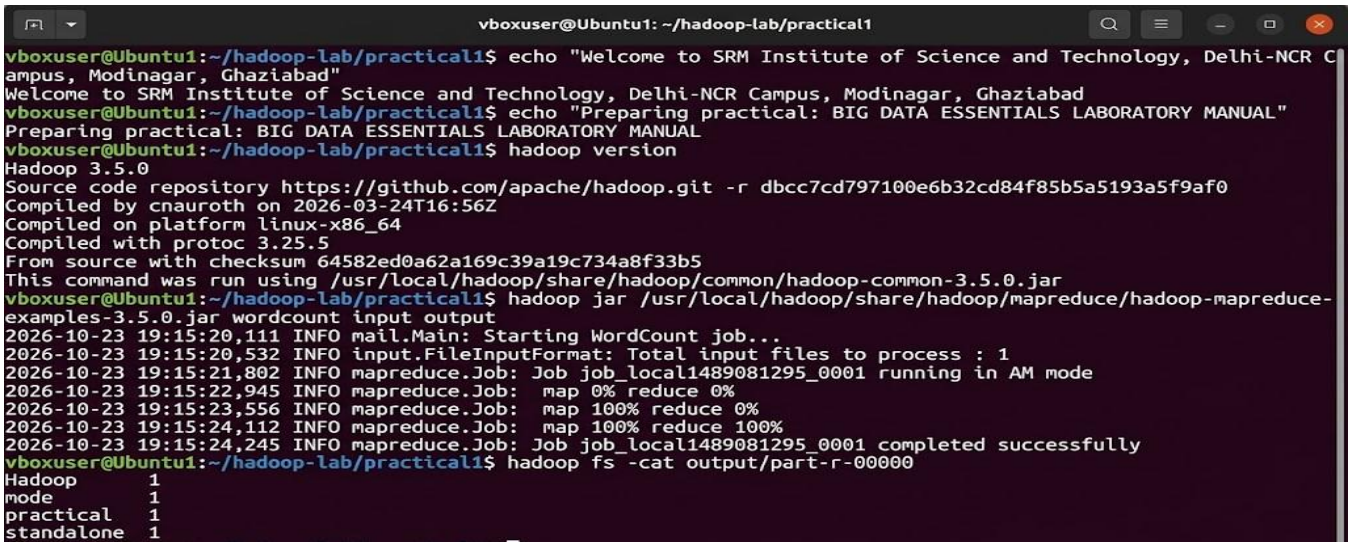
```
cat output/part-r-00000
```

Sample output:

```
Hadoop      1
mode        1
practical   1
standalone  1
```

Result

The Hadoop standalone installation was verified successfully, and a local Hadoop MapReduce application (WordCount) was executed to confirm the setup is functional.



Practical 2 — Perform Complete File and Directory Operations Using Hadoop's Filesystem Shell

Aim

To perform every essential file and directory operation — create, list, upload, download, copy, move/rename, append, and delete — using Hadoop's filesystem command-line shell (hadoop fs).

Learning Objectives

- Create directories and files using the Hadoop filesystem shell.
- Upload (put/copyFromLocal) files from the local disk into the Hadoop filesystem.
- Download (get/copyToLocal) files from the Hadoop filesystem back to the local disk.
- Copy and move/rename files and directories within the Hadoop filesystem.
- Append data to an existing file and inspect file metadata.
- Delete individual files and entire directories safely.

Requirements

- Completed Practical 1
- Working directory permissions on the local disk

Theory

Hadoop's filesystem shell (hadoop fs) provides a Unix-like interface for managing files, regardless of whether the underlying filesystem is the local filesystem (file:///, used in standalone mode) or HDFS (used once Pseudo-Distributed or Fully-Distributed mode is configured, see Part D). Because the commands are identical in both cases, everything practiced here transfers directly to a real HDFS cluster later.

In this practical, 'upload' means copying a file FROM the local operating-system filesystem INTO the Hadoop-managed filesystem, and 'download' means the reverse. In standalone mode both sides may physically be the same disk, but the commands and workflow are exactly what a real cluster requires.

Procedure — Commands to Execute

Step 1: Set up the local working directory

This is an ordinary Linux directory, separate from anything managed by Hadoop's shell.

```
mkdir -p ~/hadoop-lab/practical2/local_source
cd ~/hadoop-lab/practical2
```

Step 2: Create a local file to work with

```
echo "Welcome to Hadoop File Operations Practical" > local_source/sample.txt
echo "This is the second line of the sample file." >> local_source/sample.txt
cat local_source/sample.txt
```

Step 3: OPERATION 1 — Create a directory (hadoop fs -mkdir)

Create a Hadoop-managed directory that will hold our files, plus a nested sub-directory using -p.

```
hadoop fs -mkdir student
```

```
hadoop fs -mkdir -p student/input/raw
hadoop fs -ls student
```

Sample output:

```
Found 1 items
drwxr-xr-x   -  user  group   0  2026-08-21 10:00  student/input
```

Step 4: OPERATION 2 — Create an empty file directly (hadoop fs -touchz)

```
hadoop fs -touchz student/empty.txt
hadoop fs -ls student
```

Note: -touchz creates a zero-byte file directly inside the Hadoop filesystem, without needing a local source file.

Step 5: OPERATION 3 — Upload a file from local disk (hadoop fs -put / -copyFromLocal)

This copies local_source/sample.txt from the local filesystem into the Hadoop filesystem under student/input/.

```
hadoop fs -put local_source/sample.txt student/input/
hadoop fs -copyFromLocal local_source/sample.txt student/input/sample_copyfromlocal.txt
hadoop fs -ls student/input
```

Sample output:

```
Found 2 items
-rw-r--r--   1  user  group   93  2026-08-21 10:01  student/input/sample.txt
-rw-r--r--   1  user  group   93  2026-08-21 10:01  student/input/sample_copyfromlocal.txt
```

Step 6: OPERATION 4 — List and display uploaded files

```
hadoop fs -ls -R student
hadoop fs -cat student/input/sample.txt
```

Step 7: OPERATION 5 — Copy a file within the Hadoop filesystem (hadoop fs -cp)

```
hadoop fs -cp student/input/sample.txt student/input/sample_copy.txt
hadoop fs -ls student/input
```

Step 8: OPERATION 6 — Move / rename a file within the Hadoop filesystem (hadoop fs -mv)

hadoop fs -mv can rename a file in place, or move it into a different directory. The destination directory must already exist before moving a file into it.

```
hadoop fs -mv student/input/sample_copy.txt student/input/sample_renamed.txt
hadoop fs -mkdir -p student/archive
hadoop fs -mv student/input/sample_renamed.txt student/archive/sample_renamed.txt
hadoop fs -ls -R student
```

Note: hadoop fs -mv fails if the destination directory does not exist yet — that is why student/archive is created with -mkdir before moving the file into it.

Step 9: OPERATION 7 — Append data to an existing file (hadoop fs -appendToFile)

```
echo "This line was appended from the local disk." > local_source/extra.txt
hadoop fs -appendToFile local_source/extra.txt student/input/sample.txt
hadoop fs -cat student/input/sample.txt
```

Step 10: OPERATION 8 — Inspect file metadata (size, count, checksum)

```
hadoop fs -du -h student
hadoop fs -count student
hadoop fs -stat "%n  %o  %r" student/input/sample.txt
```

Step 11: OPERATION 9 — Download a file to the local disk (hadoop fs -get / -copyToLocal)

This copies a file FROM the Hadoop filesystem back to an ordinary local directory.

```
mkdir -p ~/hadoop-lab/practical2/local_downloads
hadoop fs -get student/input/sample.txt local_downloads/
hadoop fs -copyToLocal student/archive/sample_renamed.txt local_downloads/
ls -l local_downloads/
cat local_downloads/sample.txt
```

Step 12: OPERATION 10 — Delete a single file (hadoop fs -rm)

```
hadoop fs -rm student/empty.txt
hadoop fs -ls student
```

Step 13: OPERATION 11 — Delete a directory recursively (hadoop fs -rm -r)

Removes an entire directory and everything inside it. Use with care.

```
hadoop fs -rm -r student/archive
hadoop fs -ls -R student
```

Note: Always double-check the path before running -rm -r. There is no Recycle Bin in standalone mode unless the fs.trash.interval property is configured.

Step 14: Final cleanup (optional)

Remove everything created in this practical once it has been verified and recorded.

```
hadoop fs -rm -r student
```

Result

All essential Hadoop filesystem operations — create directory, create file, upload, list, copy, move/rename, append, inspect metadata, download, and delete (file and directory) — were performed successfully using the hadoop fs shell.

```
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -touchz student/empty.txt
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -put local_source/sample.txt student/input/
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -ls student/input
Found 1 items
-rw-r--r--   1 vboxuser vboxuser         93 2026-08-21 10:01 student/input/sample.txt
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -cp student/input/sample.txt student/input/copy.t
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -mv student/input/copy.txt student/input/renamed.
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -appendToFile local_source/extra.txt student/inpu
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -stat "%n %o %r" student/input/sample.txt
sample.txt 134217728 1
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -count student
           3            3                228 student
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -du -h student/input
135  student/input/sample.txt
93   student/input/renamed.txt
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -get student/input/sample.txt local_downloads/
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -rm student/empty.txt
Deleted student/empty.txt
vboxuser@Ubuntu1:~/hadoop-lab/practical2$ hadoop fs -rm -r student/input/raw
Deleted student/input/raw
vboxuser@Ubuntu1:~/hadoop-lab/practical2$
```

Practical 3 — Execute Word Count Using Hadoop's Built-in MapReduce Example

Aim

To execute the classic Word Count MapReduce program in Hadoop standalone mode using the bundled examples JAR.

Theory

Word Count is the canonical MapReduce example. The Mapper tokenizes each line into words and emits (word, 1) for every occurrence. The Shuffle/Sort phase groups all values by identical key (word). The Reducer sums the values for each key to produce the final word frequency.

Algorithm

1. Read the input file line by line in the Mapper.
2. Split each line into words and emit (word, 1) for each word.
3. Hadoop shuffles and sorts all intermediate pairs by key.
4. The Reducer receives each unique word with a list of 1s and sums them.
5. Write the final (word, total_count) pairs to the output directory.

Procedure — Commands to Execute

Step 1: Create the input directory and file

```
mkdir -p ~/hadoop-lab/practical3/input
cd ~/hadoop-lab/practical3
cat > input/data.txt <<'EOF'
Hadoop is powerful
Hadoop is scalable
Big data uses Hadoop
EOF
```

Step 2: Locate the Hadoop examples JAR

```
find "$HADOOP_HOME" -name "hadoop-mapreduce-examples-*.jar"
```

Step 3: Run the WordCount job

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount input
output
```

Step 4: View the result

```
cat output/part-r-00000
```

Sample output:

```
Big     1
Hadoop  3
data    1
is      2
powerful  1
scalable  1
```

```
uses    1
```

Step 5: Re-run the job (if needed)

Hadoop refuses to overwrite an existing output directory, so remove it before rerunning.

```
rm -rf output
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount input
output
```

Sample Input

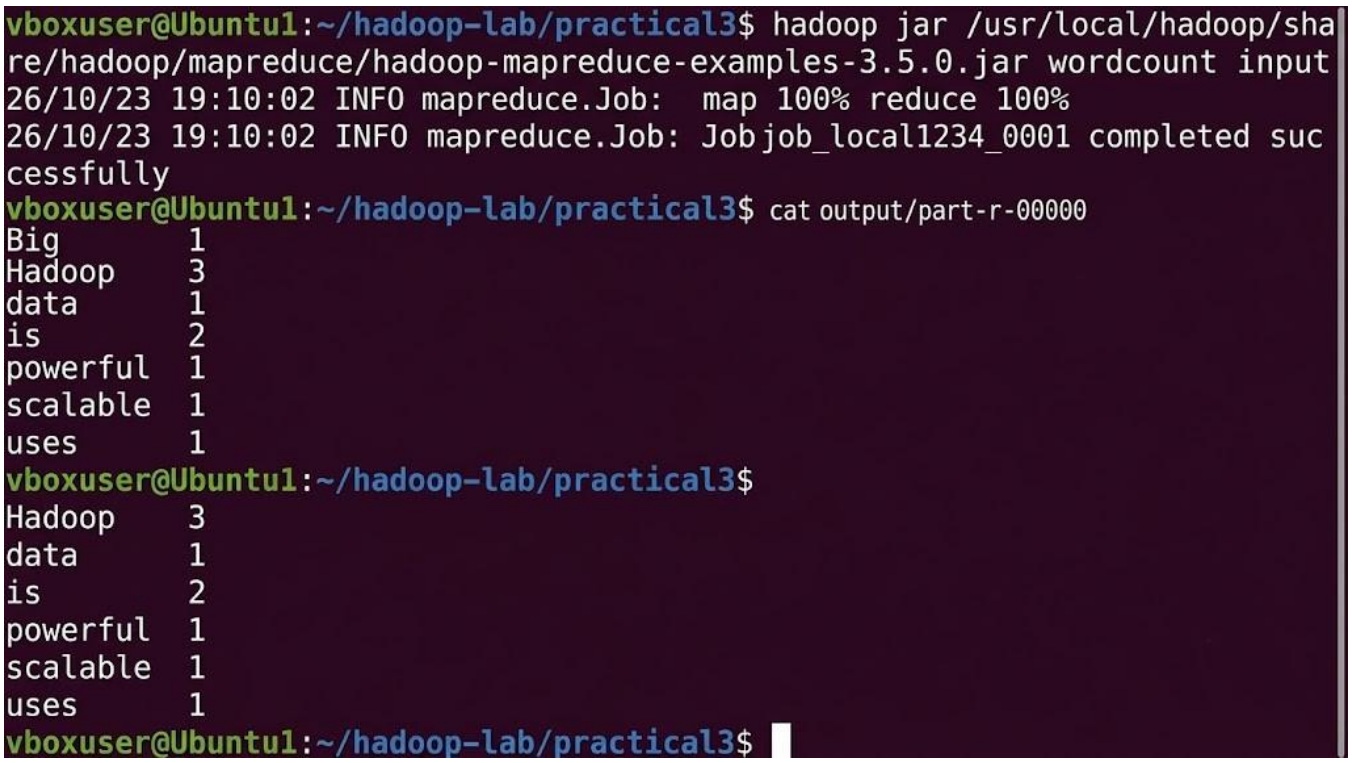
```
Hadoop is powerful
Hadoop is scalable
Big data uses Hadoop
```

Expected Output

```
Big     1
Hadoop  3
data    1
is      2
powerful  1
scalable  1
uses    1
```

Result

The Word Count MapReduce program was executed successfully in Hadoop standalone mode using the built-in examples JAR.



Practical 4 — Understand and Demonstrate MapReduce Mapper, Shuffle/Sort, and Reducer Stages

Aim

To understand the three major stages of a MapReduce job — Map, Shuffle/Sort, and Reduce — by manually tracing a small example and then verifying it against a real Hadoop run.

Theory

Every MapReduce job passes data through three conceptual stages. The Mapper converts each input record into zero or more intermediate (key, value) pairs. The Shuffle/Sort phase, managed entirely by the Hadoop framework, groups all values belonging to the same key together and sorts the keys. The Reducer then receives each unique key along with the full list of its values and produces the final output.

Algorithm

1. Split input text into individual words in the Mapper; emit (word, 1) for each.
2. Framework groups all 1s under each unique word (Shuffle/Sort).
3. Reducer iterates the list of 1s per word and sums them into a final count.

Procedure — Commands to Execute

Step 1: Trace the stages manually on paper / whiteboard

Given the input below, students should write out the Mapper output, the Shuffle/Sort grouping, and the Reducer output before running any code.

Step 2: Prepare the same input as a real file

```
mkdir -p ~/hadoop-lab/practical4/input
cd ~/hadoop-lab/practical4
cat > input/data.txt <<'EOF'
cat dog
dog cow
EOF
```

Step 3: Run WordCount to verify the manual trace

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount input
output
cat output/part-r-00000
```

Sample output:

```
cat  1
cow  1
dog  2
```

Step 4: Compare the manual trace with the actual output

Confirm that the Mapper → Shuffle/Sort → Reducer trace produced on paper matches the framework's real output.

Sample Input

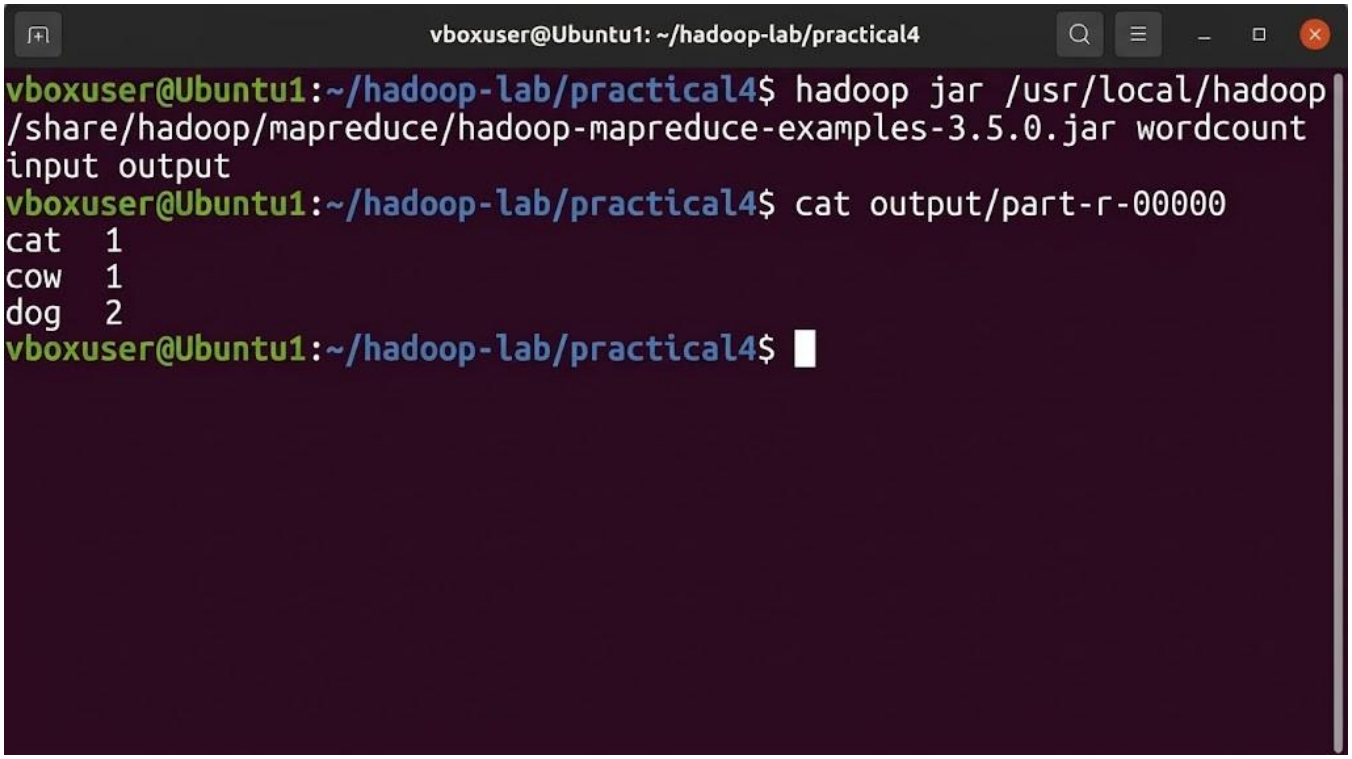
```
cat dog
dog cow
```

Expected Output

```
Mapper output:  (cat,1) (dog,1) (dog,1) (cow,1)
Shuffle/Sort :  cat->[1]   cow->[1]   dog->[1,1]
Reducer output: cat 1   cow 1   dog 2
```

Result

Students traced and verified how input records move through the Mapper, Shuffle/Sort, and Reducer stages of a MapReduce job.



Practical 5 — Run Hadoop's Built-in Examples — Grep and Pi Estimation

Aim

To discover and execute Hadoop's bundled MapReduce example applications beyond WordCount, using the Grep pattern-search example and the Pi estimation example.

Theory

The Hadoop examples JAR bundles several ready-to-run MapReduce applications besides WordCount, including grep (regex pattern counting), pi (Monte-Carlo estimation of Pi using MapReduce), wordmean, wordmedian, randomwriter, and terasort. Listing the JAR's available programs (by running it with no arguments) lets students discover exactly which examples their installed Hadoop version ships with, rather than assuming a fixed list.

Procedure — Commands to Execute

Step 1: List the examples bundled in the JAR

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar
```

Sample output:

```
An example program must be given as the first argument.
Valid program names are:
  grep: A map/reduce program that counts the matches of a regex in the input.
  pi: A map/reduce program that estimates Pi using a quasi-Monte Carlo method.
  wordcount: A map/reduce program that counts the words in the input files.
  wordmean, wordmedian, wordstandarddeviation, randomwriter, sort, terasort, ...
```

Step 2: Prepare input for the Grep example

```
mkdir -p ~/hadoop-lab/practical5/input
cd ~/hadoop-lab/practical5
cat > input/data.txt <<'EOF'
Hadoop makes big data processing simple.
Big data needs Hadoop and distributed storage.
Data engineers use Hadoop for large-scale analytics.
EOF
```

Step 3: Run the Grep example to count occurrences of 'Hadoop'

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar grep input
output_grep 'Hadoop'
```

Step 4: View the Grep result

```
cat output_grep/part-r-00000
```

Sample output:

```
3       Hadoop
```

Step 5: Run the Pi estimation example

Arguments are: number of Map tasks, and number of samples per Map task.

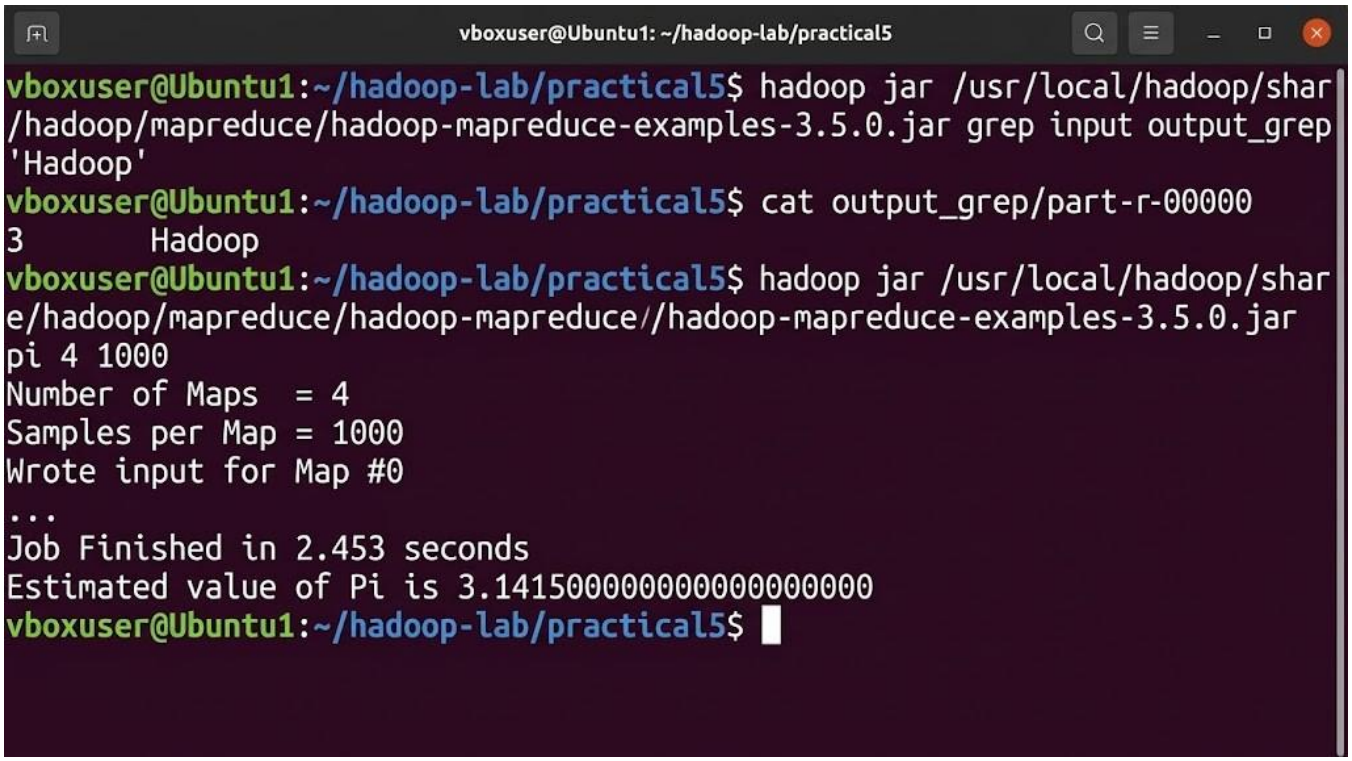
```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar pi 4 1000
```

Sample output:

```
Number of Maps  = 4
Samples per Map = 1000
...
Estimated value of Pi is 3.14150000000000000000
```

Result

Hadoop's built-in Grep and Pi-estimation MapReduce example programs were located, executed, and verified successfully in standalone mode.



Practical 6 — Run a Custom Java MapReduce Word Count Program

Aim

To write, compile, package, and execute a custom (hand-written) MapReduce Word Count program, rather than relying on the bundled example.

Learning Objectives

- Understand the structure of a MapReduce program: Mapper, Reducer, and Driver.
- Compile Java code against Hadoop's classpath.
- Package compiled classes into a runnable JAR.
- Execute the custom JAR using hadoop jar.

Algorithm

1. Mapper reads each line, splits it into tokens, and emits (word, 1) for each token.
2. Reducer receives each word with its list of counts and sums them.
3. Driver (main method) configures the Job, sets Mapper/Reducer classes, and submits the job.

Source Code — WordCount.java

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private final Text word = new Text();

        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            String[] words = value.toString().split("\\s+");

            for (String w : words) {
                if (!w.isEmpty()) {
                    word.set(w);
                    context.write(word, one);
                }
            }
        }
    }
```

```
    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values,
                           Context context)
                throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical6/input
cd ~/hadoop-lab/practical6
cat > input/data.txt <<'EOF'
Hadoop is a framework
Hadoop supports big data
EOF
```

Step 2: Save the source file

Create WordCount.java in ~/hadoop-lab/practical6/ using a text editor and paste the source code shown above.

Step 3: Compile the program against Hadoop's classpath

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes WordCount.java
```

Step 4: Package the compiled classes into a JAR

```
jar -cvf wordcount.jar -C classes .
```

Step 5: Run the custom JAR

```
hadoop jar wordcount.jar WordCount input output
```

Step 6: Display the output

```
cat output/part-r-00000
```

Sample output:

```
Hadoop  2
a       1
big     1
data    1
framework  1
is      1
supports  1
```

Sample Input

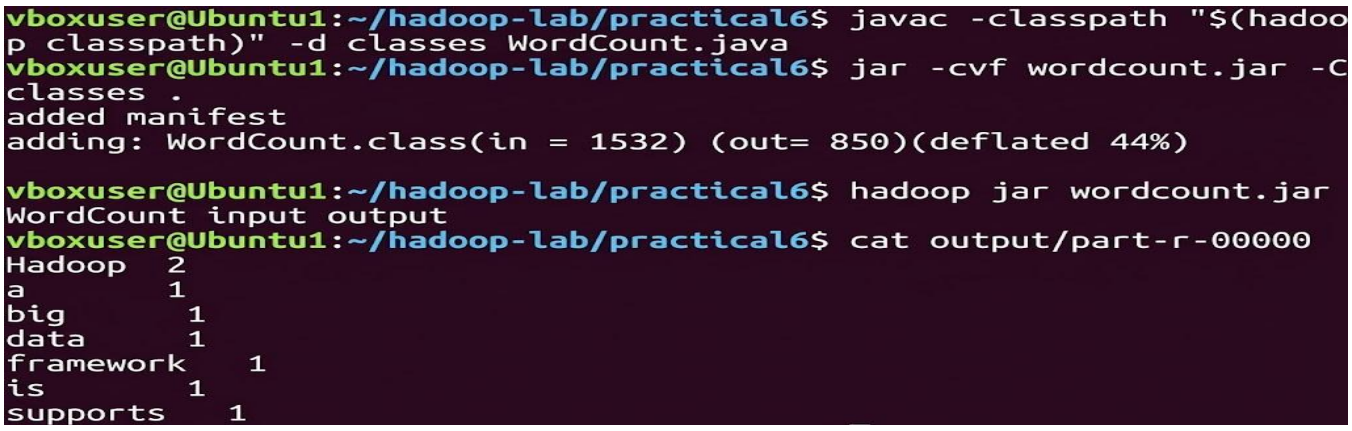
```
Hadoop is a framework
Hadoop supports big data
```

Expected Output

```
Hadoop  2
a       1
big     1
data    1
framework  1
is      1
supports  1
```

Result

A custom Java MapReduce Word Count program was successfully written, compiled, packaged, and executed in Hadoop standalone mode.



Practical 7 — Count the Number of Lines, Words, and Characters in a File

Aim

To write and execute a MapReduce program that computes the total number of lines, words, and characters in a text file.

Theory

This program demonstrates that MapReduce is not limited to simple counting of a single entity — a single Mapper can emit multiple different statistic keys (LINES, WORDS, CHARACTERS) from the same input, and a shared Reducer aggregates each independently.

Algorithm

1. For every input line, the Mapper emits ("LINES", 1).
2. The Mapper splits the line into words and emits ("WORDS", word_count_in_this_line).
3. The Mapper emits ("CHARACTERS", character_count_in_this_line).
4. The Reducer sums all values received for each of the three keys.

Source Code — LineWordCharCount.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LineWordCharCount {

    public static class StatsMapper
            extends Mapper<LongWritable, Text, Text, LongWritable> {

        private final static LongWritable ONE = new LongWritable(1);
        private Text statKey = new Text();

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            statKey.set("LINES");
            context.write(statKey, ONE);

            String[] words = line.trim().isEmpty()
                    ? new String[0] : line.trim().split("\\s+");
            statKey.set("WORDS");
            context.write(statKey, new LongWritable(words.length));

            statKey.set("CHARACTERS");
```

```
            context.write(statKey, new LongWritable(line.length()));
        }
    }

    public static class StatsReducer
            extends Reducer<Text, LongWritable, Text, LongWritable> {

        public void reduce(Text key, Iterable<LongWritable> values, Context context)
                throws IOException, InterruptedException {

            long sum = 0;
            for (LongWritable val : values) {
                sum += val.get();
            }
            context.write(key, new LongWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "line word character count");

        job.setJarByClass(LineWordCharCount.class);
        job.setMapperClass(StatsMapper.class);
        job.setReducerClass(StatsReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical7/input
cd ~/hadoop-lab/practical7
cat > input/data.txt <<'EOF'
Hadoop is useful.
MapReduce processes data.
EOF
```

Step 2: Save the source file

Create LineWordCharCount.java in ~/hadoop-lab/practical7/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes LineWordCharCount.java
```

```
jar -cvf lwc.jar -C classes .
hadoop jar lwc.jar LineWordCharCount input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
CHARACTERS  42
LINES       2
WORDS       6
```

Sample Input

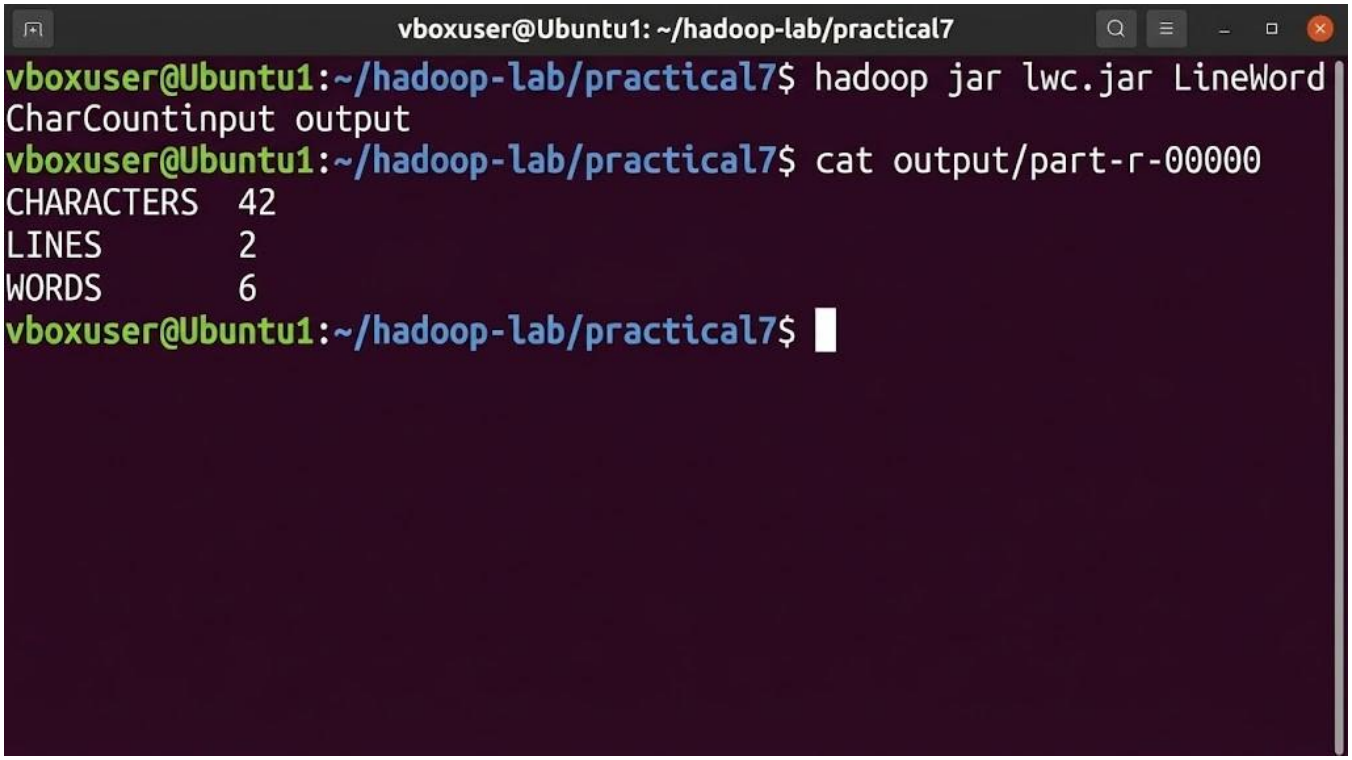
```
Hadoop is useful.
MapReduce processes data.
```

Expected Output

```
CHARACTERS  42
LINES       2
WORDS       6
```

Result

A MapReduce program was implemented and executed to compute the total number of lines, words, and characters in a text file.



Practical 8 — Find the Frequency of Each Character Using MapReduce

Aim

To write and execute a MapReduce program that computes the frequency of every character appearing in a text file, ignoring whitespace.

Algorithm

1. Mapper reads each line and iterates over every character.
2. For every non-whitespace character, the Mapper emits (character, 1).
3. The Reducer sums the counts received for each unique character.

Source Code — CharFrequency.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class CharFrequency {

    public static class CharMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text charKey = new Text();

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            for (char c : line.toCharArray()) {
                if (!Character.isWhitespace(c)) {
                    charKey.set(String.valueOf(c));
                    context.write(charKey, one);
                }
            }
        }
    }

    public static class CharReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
```

```
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "character frequency count");

        job.setJarByClass(CharFrequency.class);
        job.setMapperClass(CharMapper.class);
        job.setReducerClass(CharReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical8/input
cd ~/hadoop-lab/practical8
echo -n "hadoop" > input/data.txt
```

Step 2: Save the source file

Create CharFrequency.java in ~/hadoop-lab/practical8/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes CharFrequency.java
jar -cvf charfreq.jar -C classes .
hadoop jar charfreq.jar CharFrequency input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
a  1
d  1
h  1
o  2
```

```
p   1
```

Sample Input

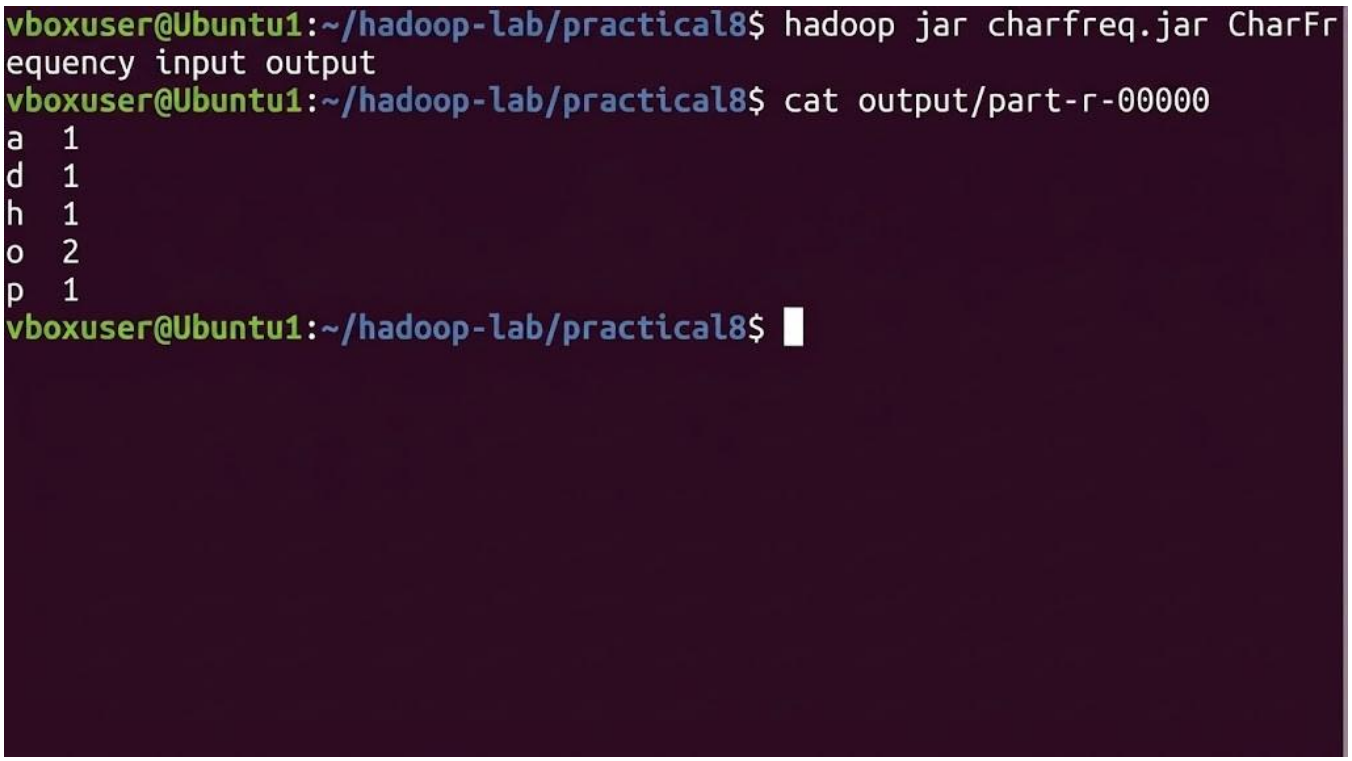
```
hadoop
```

Expected Output

```
a   1
d   1
h   1
o   2
p   1
```

Result

A MapReduce program was implemented and executed to compute the frequency of each character in a text file.



Practical 9 — Find the Maximum Marks Among Students Using MapReduce

Aim

To write and execute a MapReduce program that finds the highest mark from a list of student records.

Algorithm

1. Mapper reads each record ("rollNo mark") and emits ("Maximum", mark).
2. A Combiner (reusing the Reducer logic) pre-aggregates the maximum on each Map task, reducing data shuffled.
3. The Reducer scans all values received for the single key and keeps the largest one.

Source Code — MaxMarks.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxMarks {

    public static class MaxMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private Text outKey = new Text("Maximum");

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            String[] parts = line.split("\\s+");
            if (parts.length < 2) return;

            int mark = Integer.parseInt(parts[1]);
            context.write(outKey, new IntWritable(mark));
        }
    }

    public static class MaxReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {

            int max = Integer.MIN_VALUE;
```

```
            for (IntWritable val : values) {
                if (val.get() > max) max = val.get();
            }
            context.write(key, new IntWritable(max));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "maximum marks");

        job.setJarByClass(MaxMarks.class);
        job.setMapperClass(MaxMapper.class);
        job.setCombinerClass(MaxReducer.class);
        job.setReducerClass(MaxReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical9/input
cd ~/hadoop-lab/practical9
cat > input/marks.txt <<'EOF'
101 78
102 91
103 88
104 95
105 86
EOF
```

Step 2: Save the source file

Create MaxMarks.java in ~/hadoop-lab/practical9/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes MaxMarks.java
jar -cvf maxmarks.jar -C classes .
hadoop jar maxmarks.jar MaxMarks input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
Maximum  95
```

Sample Input

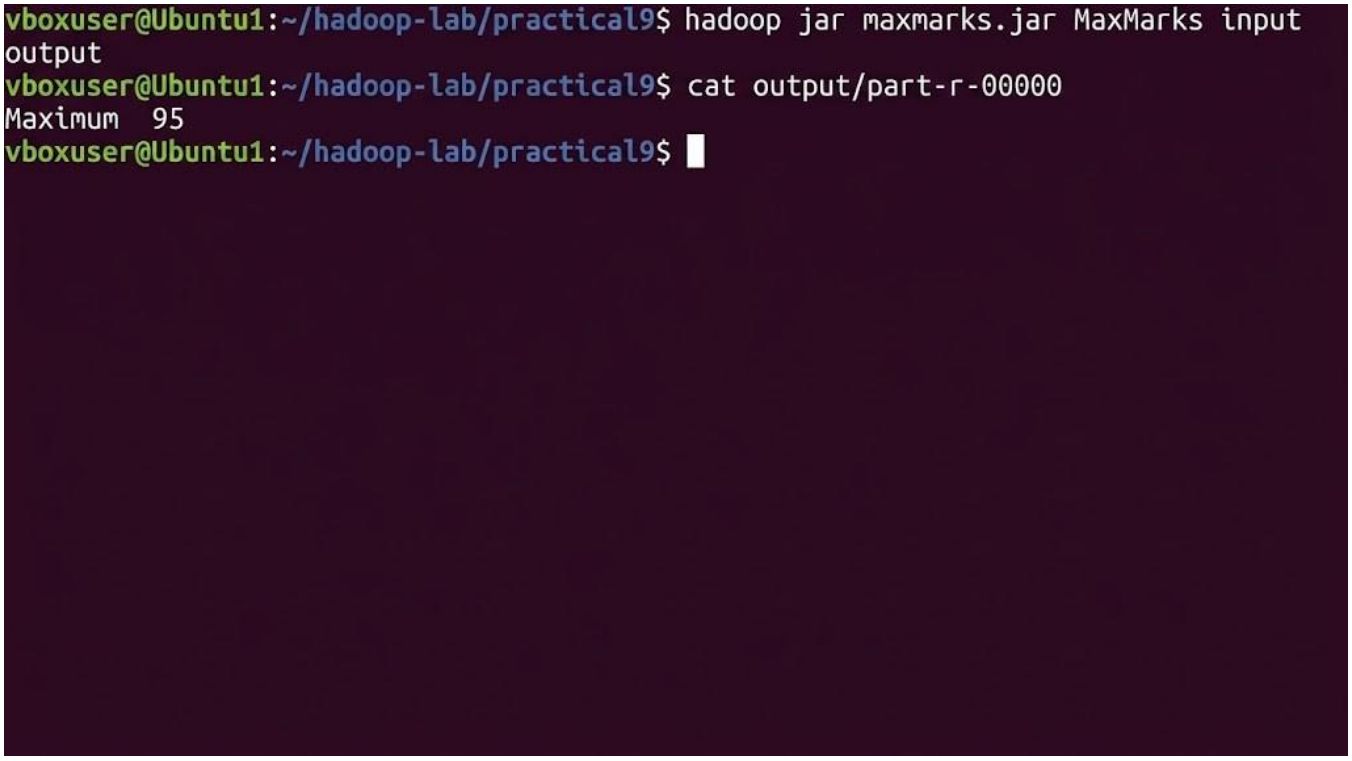
```
101 78
102 91
103 88
104 95
105 86
```

Expected Output

```
Maximum  95
```

Result

A MapReduce program was implemented and executed to find the maximum mark from a set of student records.



Practical 10 — Calculate the Average Marks of Students Using MapReduce

Aim

To write and execute a MapReduce program that calculates the average of a set of student marks.

Algorithm

1. Mapper reads each record and emits ("Marks", mark).
2. The Reducer receives every mark under the single key "Marks", accumulating both the running sum and the count as it iterates.
3. The Reducer computes average = sum / count and writes it as a DoubleWritable.

Source Code — AverageMarks.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageMarks {

    public static class AvgMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private Text outKey = new Text("Marks");

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            String[] parts = line.split("\\s+");
            if (parts.length < 2) return;

            int mark = Integer.parseInt(parts[1]);
            context.write(outKey, new IntWritable(mark));
        }
    }

    public static class AvgReducer
            extends Reducer<Text, IntWritable, Text, DoubleWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
```

```
            int sum = 0;
            int count = 0;

            for (IntWritable val : values) {
                sum += val.get();
                count++;
            }

            double average = count == 0 ? 0.0 : (double) sum / count;
            context.write(key, new DoubleWritable(average));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "average marks");

        job.setJarByClass(AverageMarks.class);
        job.setMapperClass(AvgMapper.class);
        job.setReducerClass(AvgReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical10/input
cd ~/hadoop-lab/practical10
cat > input/marks.txt <<'EOF'
101 80
102 90
103 70
104 60
EOF
```

Step 2: Save the source file

Create AverageMarks.java in ~/hadoop-lab/practical10/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes AverageMarks.java
jar -cvf avgmarks.jar -C classes .
```

```
hadoop jar avgmarks.jar AverageMarks input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
Marks  75.0
```

Sample Input

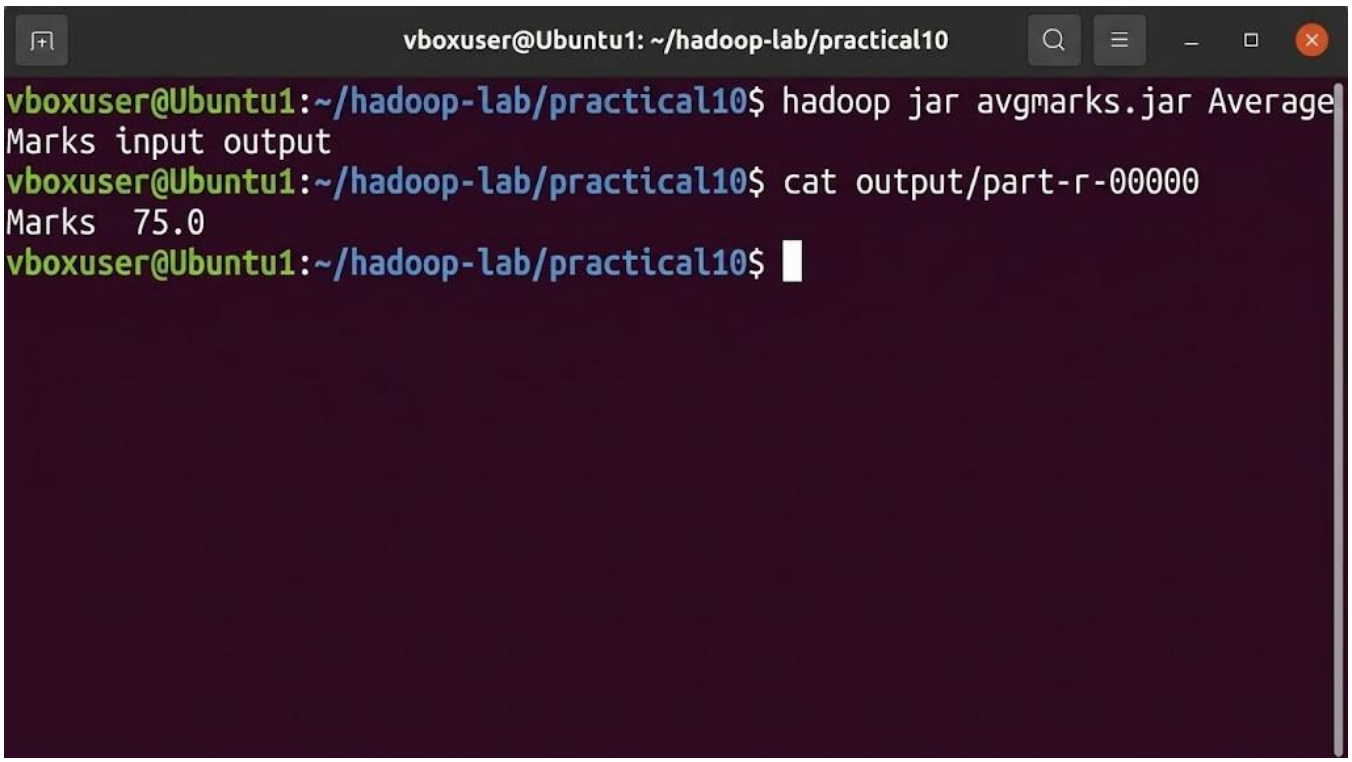
```
101 80
102 90
103 70
104 60
```

Expected Output

```
Marks  75.0
```

Result

A MapReduce program was implemented and executed to calculate the average of student marks.



Practical 11 — Sort Student Records by Marks Using MapReduce

Aim

To understand and demonstrate sorting of key-value records in MapReduce by using student marks as the intermediate key.

Theory

MapReduce automatically sorts intermediate keys during the Shuffle/Sort phase before they reach the Reducer. By making the numeric mark the MAP OUTPUT KEY (instead of the roll number), the framework performs the sorting for us — no explicit sort logic is required in the Reducer.

Algorithm

1. Mapper reads each record ("rollNo mark") and emits (mark, "rollNo mark") — i.e., the mark becomes the key.
2. Hadoop's Shuffle/Sort phase sorts all keys (marks) in ascending order.
3. A single Reducer (numReduceTasks = 1) simply re-emits each record in the sorted order it receives them.

Source Code — SortByMarks.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SortByMarks {

    public static class SortMapper
            extends Mapper<LongWritable, Text, IntWritable, Text> {

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            String[] parts = line.split("\\s+");
            if (parts.length < 2) return;

            String rollNo = parts[0];
            int mark = Integer.parseInt(parts[1]);

            context.write(new IntWritable(mark), new Text(rollNo + " " + mark));
        }
    }
```

```
    public static class SortReducer
            extends Reducer<IntWritable, Text, IntWritable, Text> {

        public void reduce(IntWritable key, Iterable<Text> values, Context context)
                throws IOException, InterruptedException {

            for (Text record : values) {
                context.write(key, record);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "sort student records by marks");

        job.setJarByClass(SortByMarks.class);
        job.setMapperClass(SortMapper.class);
        job.setReducerClass(SortReducer.class);
        job.setNumReduceTasks(1);

        job.setOutputKeyClass(IntWritable.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical11/input
cd ~/hadoop-lab/practical11
cat > input/marks.txt <<'EOF'
101 78
102 91
103 65
104 88
EOF
```

Step 2: Save the source file

Create SortByMarks.java in ~/hadoop-lab/practical11/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes SortByMarks.java
jar -cvf sortmarks.jar -C classes .
hadoop jar sortmarks.jar SortByMarks input output
```

Step 4: Display the output (sorted ascending by mark)

```
cat output/part-r-00000
```

Sample output:

```
65  103 65
78  101 78
88  104 88
91  102 91
```

Sample Input

```
101 78
102 91
103 65
104 88
```

Expected Output

```
65  103 65
78  101 78
88  104 88
91  102 91
```

Result

A MapReduce program was implemented that sorts student records in ascending order of marks by using the mark itself as the intermediate key.



Practical 12 — Filter Student Records Using MapReduce (Map-Only Job)

Aim

To write and execute a MapReduce program that filters and displays only the records satisfying a given condition (marks >= 50), using a map-only job.

Theory

Not every MapReduce job needs a Reducer. When the transformation is a pure per-record filter (keep or discard), setting job.setNumReduceTasks(0) makes the job map-only: the Mapper's output becomes the final output directly, with no Shuffle/Sort or Reduce phase — which is faster for this kind of task.

Algorithm

1. Mapper reads each record ("rollNo mark").
2. If mark >= threshold (50), the Mapper writes the original record straight to the output.
3. No Reducer is required; the job is configured with zero reduce tasks.

Source Code — FilterMarks.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class FilterMarks {

    public static class FilterMapper
            extends Mapper<LongWritable, Text, NullWritable, Text> {

        private static final int THRESHOLD = 50;

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            String[] parts = line.split("\\s+");
            if (parts.length < 2) return;

            int mark = Integer.parseInt(parts[1]);

            if (mark >= THRESHOLD) {
                context.write(NullWritable.get(), value);
            }
        }
    }
```

```
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "filter students by marks");

        job.setJarByClass(FilterMarks.class);
        job.setMapperClass(FilterMapper.class);
        job.setNumReduceTasks(0);

        job.setOutputKeyClass(NullWritable.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical12/input
cd ~/hadoop-lab/practical12
cat > input/marks.txt <<'EOF'
101 45
102 78
103 91
104 38
105 66
EOF
```

Step 2: Save the source file

Create FilterMarks.java in ~/hadoop-lab/practical12/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes FilterMarks.java
jar -cvf filtermarks.jar -C classes .
hadoop jar filtermarks.jar FilterMarks input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
102 78
103 91
105 66
```

Sample Input

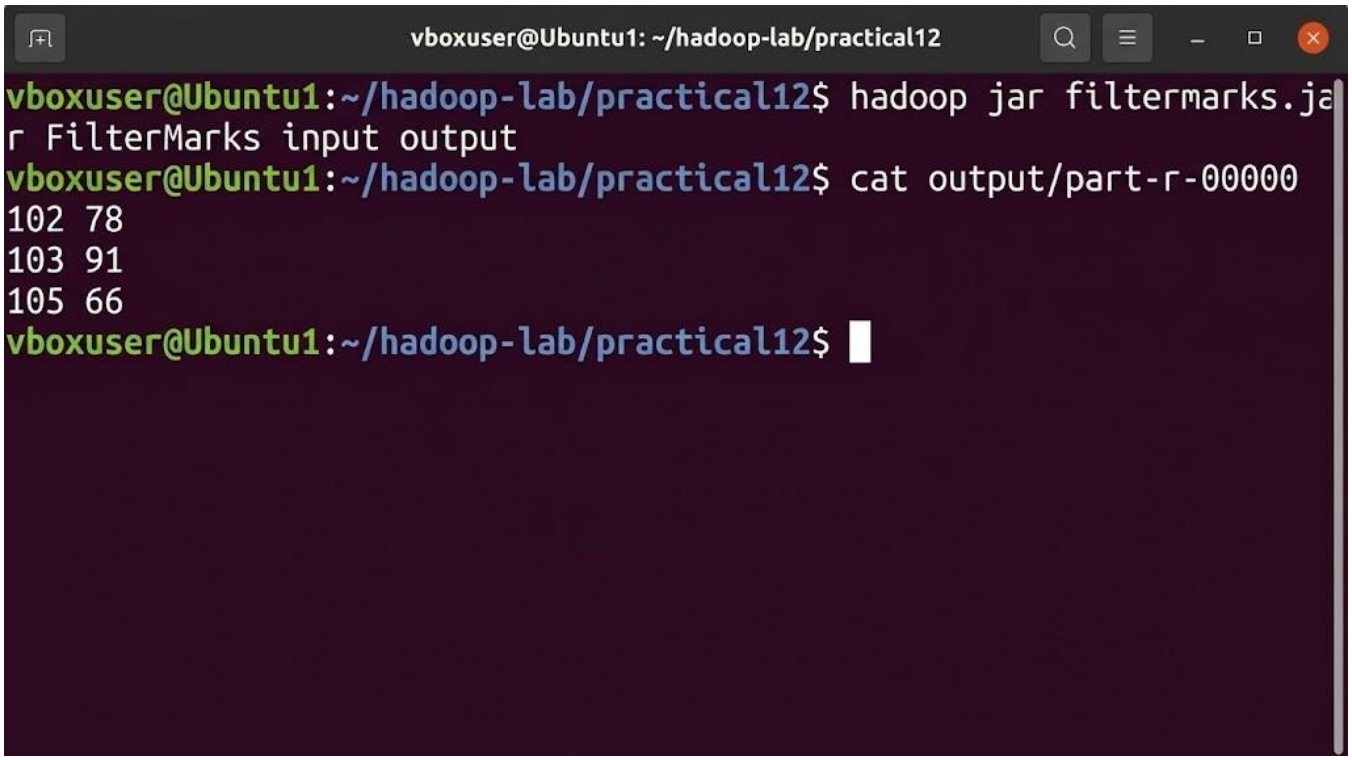
```
101 45
102 78
103 91
104 38
105 66
```

Expected Output

```
102 78
103 91
105 66
```

Result

A map-only MapReduce program was implemented and executed to filter student records with marks greater than or equal to 50.



Practical 13 — Count the Number of Students in Each Department Using MapReduce

Aim

To write and execute a MapReduce program that counts how many students belong to each department.

Algorithm

1. Mapper reads each record ("rollNo department") and emits (department, 1).
2. A Combiner pre-sums counts per department on each Map task.
3. The Reducer sums all counts received for each department key.

Source Code — DepartmentCount.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class DepartmentCount {

    public static class DeptMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text dept = new Text();

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            String[] parts = line.split("\\s+");
            if (parts.length < 2) return;

            dept.set(parts[1]);
            context.write(dept, one);
        }
    }

    public static class DeptReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
```

```
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "department wise student count");

        job.setJarByClass(DepartmentCount.class);
        job.setMapperClass(DeptMapper.class);
        job.setCombinerClass(DeptReducer.class);
        job.setReducerClass(DeptReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical13/input
cd ~/hadoop-lab/practical13
cat > input/students.txt <<'EOF'
101 CSE
102 IT
103 CSE
104 ECE
105 IT
106 CSE
EOF
```

Step 2: Save the source file

Create DepartmentCount.java in ~/hadoop-lab/practical13/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes DepartmentCount.java
jar -cvf deptcount.jar -C classes .
hadoop jar deptcount.jar DepartmentCount input output
```

Step 4: Display the output

```
cat output/part-r-00000
```

Sample output:

```
CSE  3
ECE  1
IT   2
```

Sample Input

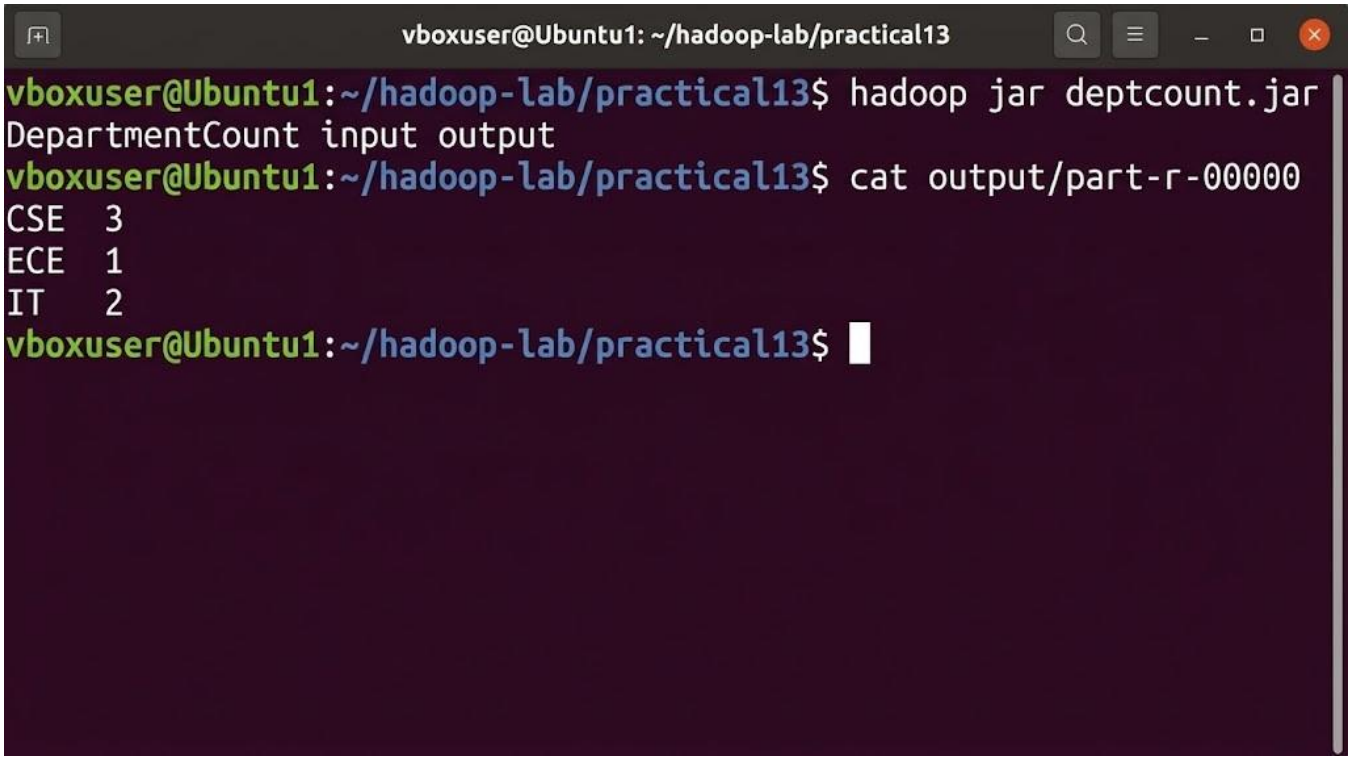
```
101 CSE
102 IT
103 CSE
104 ECE
105 IT
106 CSE
```

Expected Output

```
CSE  3
ECE  1
IT   2
```

Result

A MapReduce program was implemented and executed to count the number of students belonging to each department.



Practical 14 — Compare MapReduce Execution With a Normal Sequential Program

Aim

To compare traditional sequential word-count processing against the MapReduce programming model, both conceptually and by measured execution time.

Requirements

- Python 3 installed (python3 --version)
- The compiled wordcount.jar from Practical 6

Theory

A traditional program processes input sequentially, in one thread, on one machine — simple for small data but does not scale as data grows. MapReduce splits work into independent Map tasks that can run in parallel (across cores or machines) followed by a Shuffle/Sort and Reduce phase, trading some overhead and complexity for the ability to scale horizontally across a cluster.

On a single standalone laptop, MapReduce cannot show its true multi-node performance advantage — in fact, for small inputs, the JVM startup and framework overhead usually makes the sequential program faster. This practical is designed to make that overhead visible and understood, not to prove MapReduce is faster on one machine.

Source Code — wordcount_sequential.py

```
import sys
from collections import Counter

def main():
    if len(sys.argv) < 2:
        print("Usage: python3 wordcount_sequential.py <input_file>")
        sys.exit(1)

    counts = Counter()

    with open(sys.argv[1], "r") as f:
        for line in f:
            for word in line.strip().split():
                counts[word] += 1

    for word, count in sorted(counts.items()):
        print(f"{word}\t{count}")

if __name__ == "__main__":
    main()
```

Procedure — Commands to Execute

Step 1: Create a working directory and a larger sample file

```
mkdir -p ~/hadoop-lab/practical14
cd ~/hadoop-lab/practical14
```

```
for i in $(seq 1 20000); do echo "Hadoop enables distributed big data processing at scale"; done
> bigfile.txt
wc -l bigfile.txt
```

Step 2: Save the sequential Python program

Create wordcount_sequential.py in ~/hadoop-lab/practical14/ using the source code above.

Step 3: Run and time the sequential program

```
time python3 wordcount_sequential.py bigfile.txt > seq_output.txt
cat seq_output.txt
```

Step 4: Run and time the same job using MapReduce (reusing Practical 6's JAR)

```
mkdir -p input
cp bigfile.txt input/
time hadoop jar ~/hadoop-lab/practical6/wordcount.jar WordCount input mr_output
cat mr_output/part-r-00000
```

Step 5: Compare the two 'real' times reported and discuss

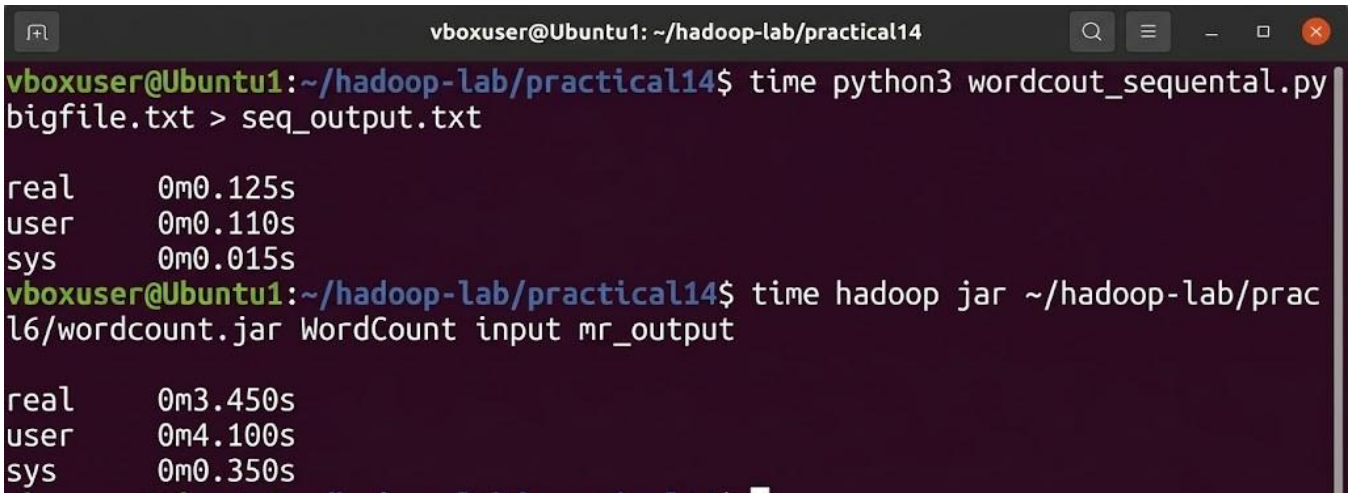
Record both wall-clock times in the lab record and discuss why the smaller-scale result differs from what would be expected on a real multi-node cluster with much larger data.

Expected Output

```
Discussion points:
  Traditional program : runs sequentially, single process, limited by one CPU core.
  MapReduce program    : splits work into Map/Reduce tasks; scales horizontally on a real
cluster.
  On one laptop with small/medium data, MapReduce's JVM startup + framework overhead often makes
  it SLOWER than the sequential program — this is expected and should be explained, not hidden.
```

Result

The execution characteristics of a traditional sequential program and an equivalent MapReduce program were compared, and the overhead of the MapReduce framework on a single standalone machine was measured and explained.



Practical 15 — Mini Project — Student Result Analysis Using MapReduce

Aim

To design and execute a complete MapReduce mini-application that analyzes student result data: highest mark, lowest mark, average mark, and department-wise student counts, in a single job.

Requirements

- Completed Practicals 9, 10, and 13 (recommended, for comparison)

Theory

This mini project combines the techniques from Practicals 9, 10, and 13 into one job. The Mapper emits two different families of keys from the same input record: a fixed "STATS" key carrying each student's mark (for max/min/average), and a "DEPT_<name>" key (for department-wise counting). A single Reducer distinguishes between the two key families and applies the appropriate aggregation logic to each.

Algorithm

1. Mapper parses each record as "rollNo department marks".
2. For every record, the Mapper emits ("STATS", marks) for global statistics.
3. For every record, the Mapper also emits ("DEPT_" + department, 1) for department counting.
4. The Reducer checks which key family it received: for "STATS" it computes max, min, and average across all values; for any "DEPT_..." key it sums the counts and emits the department name with its total.

Source Code — StudentResultAnalysis.java

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class StudentResultAnalysis {

    public static class ResultMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString().trim();
            if (line.isEmpty()) return;

            // Format: rollNo department marks
            String[] parts = line.split("\\s+");
```

```
            if (parts.length < 3) return;

            String department = parts[1];
            int marks = Integer.parseInt(parts[2]);

            outKey.set("STATS");
            context.write(outKey, new IntWritable(marks));

            outKey.set("DEPT_" + department);
            context.write(outKey, new IntWritable(1));
        }
    }

    public static class ResultReducer
            extends Reducer<Text, IntWritable, Text, Text> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {

            String keyStr = key.toString();

            if (keyStr.equals("STATS")) {
                int max = Integer.MIN_VALUE;
                int min = Integer.MAX_VALUE;
                int sum = 0;
                int count = 0;

                for (IntWritable val : values) {
                    int m = val.get();
                    max = Math.max(max, m);
                    min = Math.min(min, m);
                    sum += m;
                    count++;
                }

                double average = count == 0 ? 0.0 : (double) sum / count;

                context.write(new Text("Highest_Mark"), new Text(String.valueOf(max)));
                context.write(new Text("Lowest_Mark"), new Text(String.valueOf(min)));
                context.write(new Text("Average_Mark"), new Text(String.valueOf(average)));
            } else if (keyStr.startsWith("DEPT_")) {
                int count = 0;
                for (IntWritable val : values) {
                    count += val.get();
                }
                String department = keyStr.substring(5);
                context.write(new Text(department), new Text(String.valueOf(count)));
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "student result analysis");
```

```
        job.setJarByClass(StudentResultAnalysis.class);
        job.setMapperClass(ResultMapper.class);
        job.setReducerClass(ResultReducer.class);
        job.setNumReduceTasks(1);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Procedure — Commands to Execute

Step 1: Create the project directory and input file

```
mkdir -p ~/hadoop-lab/practical15/input
cd ~/hadoop-lab/practical15
cat > input/students.txt <<'EOF'
101 CSE 78
102 IT 91
103 CSE 65
104 ECE 88
105 IT 72
106 CSE 95
107 ECE 81
108 IT 84
EOF
```

Step 2: Save the source file

Create StudentResultAnalysis.java in ~/hadoop-lab/practical15/ using the source code above.

Step 3: Compile, package, and run

```
mkdir -p classes
javac -classpath "$(hadoop classpath)" -d classes StudentResultAnalysis.java
jar -cvf resultanalysis.jar -C classes .
hadoop jar resultanalysis.jar StudentResultAnalysis input output
```

Step 4: Display the combined output

```
cat output/part-r-00000
```

Step 5: (Optional extension) List students scoring 80 or above

Reuse the FilterMarks program from Practical 12 with THRESHOLD changed to 80, run against the same input format (rollNo marks — extract columns 1 and 3 first if needed), to produce the 'scored >= 80' list as a separate output.

Sample Input

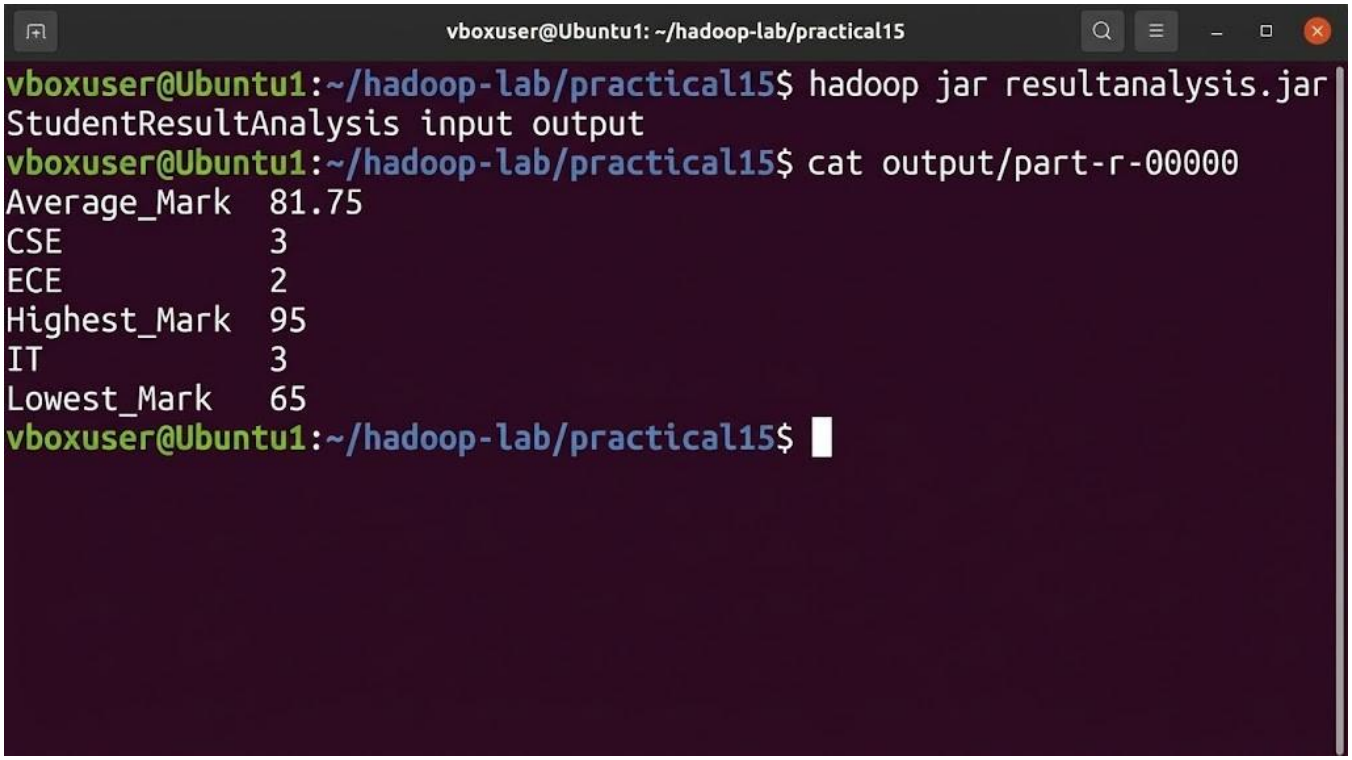
```
101 CSE 78
102 IT 91
103 CSE 65
104 ECE 88
105 IT 72
106 CSE 95
107 ECE 81
108 IT 84
```

Expected Output

```
Average_Mark  81.75
CSE           3
ECE           2
Highest_Mark  95
IT            3
Lowest_Mark   65
```

Result

A complete MapReduce mini-application was designed and executed for student result analysis, computing the highest, lowest, and average marks together with department-wise student counts in a single job.



Part C — Optional Transition From Standalone to Pseudo-Distributed Mode

Why is this needed?

If the course syllabus specifically requires HDFS, NameNode/DataNode behaviour, replication, or true hdfs dfs commands, pure standalone mode is not sufficient — those commands only operate meaningfully once HDFS daemons are actually running.

```
hdfs dfs -put
hdfs dfs -ls
hdfs dfs -mkdir
```

In that case, Hadoop should be configured in Pseudo-Distributed Mode on the same laptop. In pseudo-distributed mode, Hadoop's services run as separate Java processes on one machine — this is the standard way to teach real HDFS and YARN concepts without needing multiple physical computers.

Typical services running in pseudo-distributed mode

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
```

Exactly which services are required depends on whether the practical needs HDFS only, YARN only, or both.

Note: Do not run 'hdfs namenode -format' repeatedly. Formatting the NameNode re-initializes its filesystem metadata and can make any previously stored HDFS data inaccessible. Configure the required core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml files first, and follow the setup procedure that matches your exact Hadoop version before formatting.

Part D — Basic HDFS Commands After Pseudo-Distributed Configuration

Once HDFS is actually configured and its daemons are running (verified with jps), the following true HDFS commands become meaningful and map directly onto the file operations already practiced in Practical 2.

Check running Java processes

```
jps
```

Expected services include:

```
NameNode
DataNode
SecondaryNameNode
Jps
```

Operation	Command
Create HDFS directory	`hdfs dfs -mkdir -p /student/input`
Upload file	`hdfs dfs -put sample.txt /student/input/`
List HDFS files	`hdfs dfs -ls /student/input`
Read file	`hdfs dfs -cat /student/input/sample.txt`
Copy file	`hdfs dfs -cp /student/input/sample.txt /student/input/copy.txt`
Rename file	`hdfs dfs -mv /student/input/copy.txt /student/input/newname.txt`
Download file	`hdfs dfs -get /student/input/newname.txt .`
Delete file	`hdfs dfs -rm /student/input/newname.txt`
Delete directory	`hdfs dfs -rm -r /student`
Check HDFS storage	`hdfs dfs -df -h`

Part E — Suggested Practical Sequence for Engineering Students

#	Topic	Main command / tool
1	Verify Hadoop standalone installation	hadoop version, hadoop jar
2	Complete Hadoop filesystem operations (create, copy, move, upload, download, delete)	hadoop fs
3	Word Count using built-in MapReduce	hadoop jar
4	Mapper, Shuffle/Sort, and Reducer stages	MapReduce
5	Built-in examples: Grep and Pi estimation	hadoop jar
6	Custom Java WordCount	javac, jar, hadoop jar
7	Line, word, and character statistics	MapReduce
8	Character frequency	MapReduce
9	Maximum marks	MapReduce
10	Average marks	MapReduce
11	Sorting records by marks	MapReduce
12	Filtering records (map-only job)	MapReduce
13	Department-wise counting	MapReduce
14	Sequential vs. MapReduce comparison	python3, hadoop jar
15	Student result mini project	MapReduce

Part F — Common Errors and Fixes

Error 1: 'hadoop: command not found'

```
echo $HADOOP_HOME
echo $PATH
```

Make sure Hadoop's bin and sbin directories are included in PATH, then re-source your shell profile (source ~/.bashrc).

Error 2: 'JAVA_HOME is not set'

```
java -version
echo $JAVA_HOME
```

Configure JAVA_HOME to point to the installed JDK directory, then re-source your shell profile.

Error 3: Output directory already exists

```
rm -rf output
# For HDFS:
hdfs dfs -rm -r /output
```

MapReduce refuses to write into an output directory that already exists. Only delete it once you are sure it does not contain data you still need.

Error 4: Examples JAR not found

```
find "$HADOOP_HOME" -name "hadoop-mapreduce-examples-*.jar"
```

If no file is returned, your Hadoop installation may not include the examples JAR — check the share/hadoop/mapreduce directory manually.

Error 5: ClassNotFoundException when running a custom JAR

```
hadoop classpath
javac -classpath "$(hadoop classpath)" -d classes YourProgram.java
```

Always compile custom Java MapReduce programs against Hadoop's classpath, and double-check the class name passed to 'hadoop jar yourjar.jar ClassName' matches exactly (including case).

Error 6: NumberFormatException while parsing marks

```
cat -A input/marks.txt | head
```

This usually means the input file has extra spaces, a trailing blank line, or a header row. Trim each line and skip empty lines before calling Integer.parseInt(), as done throughout Practicals 9–13, 15.